

Solutions to FIT3080 Applied Class on Problem Solving as Search: Problem Formulation, Backtrack, Tree Search and Graphsearch

Exercise 1: Backtrack

Specify a state representation, operators and goal test to solve the missionaries and cannibals problem:

Three missionaries and three cannibals come to a river. There is a boat on their side of the river that can be used either by one or two persons. How should they use this boat to cross the river in such a way that cannibals never outnumber missionaries on either side of the river?

- (a) Illustrate the workings of the backtrack algorithm (Backtrack or Backtrack1) to solve this problem. To expedite your solution, you may want to specify a heuristic function over the actions for different states.
- (b) [Optional] Write a python program to implement your backtrack solution.

SOLUTION:

The main idea of the backtrack algorithm is that it selects **one successor** for a parent node, and only considers that particular successor, then, in turn, it selects a successor for it, etc. When a successor can not be selected, then (and only then), another alternative is attempted. At any point in time, the algorithm is aware of only one path from a particular node (which is the node under consideration) to the start node.

Now, let us consider the solution:

State Representation

- ($\#$ missionaries at start, $\#$ cannibals at start) and total number of missionaries and cannibals.
- B – has value 0 if boat is at the start, and value 1 if boat is at the finish.
- i – number of times the boat has crossed the river.

Note that a database containing only the number of cannibals and missionaries on one side of the river is not enough, since the algorithm would not know what are the conditions and the other side, e.g., whether a constraint has been violated, whether the termination condition has been fulfilled, etc. That's why we also need the total.

Operators

R_{ij} – put in the boat i missionaries and j cannibals, where $0 < i + j \leq 2$.

Rules of life which govern this process:

- The boat cannot travel when it is unmanned.
- The boat will travel when there is at least one person in it (it will, however, wait for an additional passenger, if the intent is to send two people).

- Person(s) in the boat automatically disembark at the opposite side of the river.

Constraints

The cannibals can never outnumber the missionaries on either side of the river. For $c = \#$ of cannibals on the left bank, and $m = \#$ of missionaries on the left bank, the constraints are: $[(c \leq m) \wedge (3 - c \leq 3 - m)] \vee (m = 3) \vee m = 0$. We can collapse these inequalities to: $(c = m) \vee (m = 0) \vee (m = 3)$.

Goal Test

$$\# \text{-of-cannibals}(\text{start}) + \# \text{-of-missionaries}(\text{start}) = 0$$

Note that the termination condition can **not** consider just the cannibals or just the missionaries, because then the algorithm would stop with 3 cannibals or 3 missionaries on the finishing river bank, and some people could be left on the starting bank.

Heuristic Function

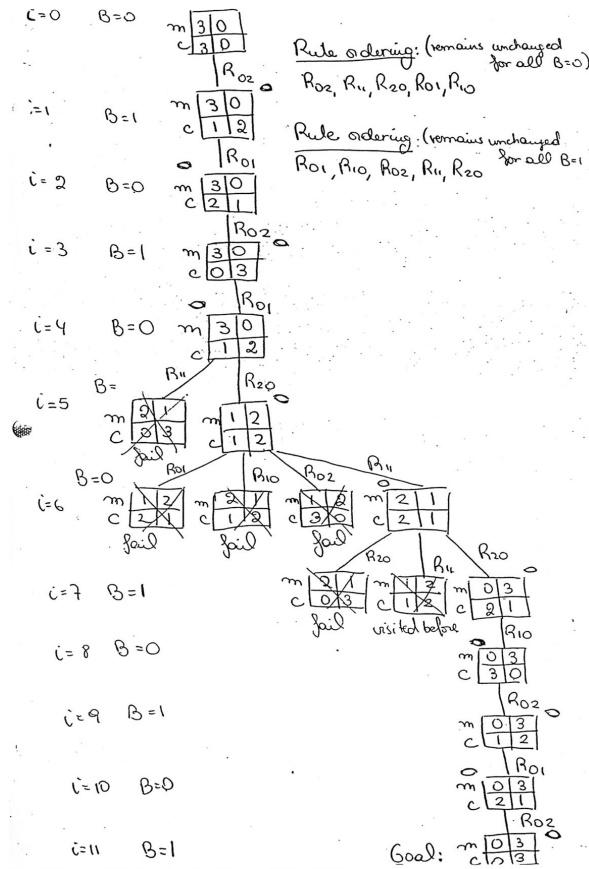
$$\# \text{-of-cannibals}(\text{finish}) + \# \text{-of-missionaries}(\text{finish})$$

This number ought to be maximized.

Tie-breaking rule (arbitrary): Take the cannibals first.

(a) Backtrack Solution

When backtracking, keep in mind that the applicable rules are ordered either arbitrarily or according to heuristic merit. This is done **once** for each recursive call to Backtrack (or Backtrack1). After the ordering has been established, the first rule is attempted, if it fails, then the next one is tried, and so on.



(b) Python program: See associated program

Exercise 2: Backtrack, Constraint Satisfaction

Consider a problem where one has to insert the numbers 1, 2, 3, 4, 5 and 6 in the perimeter of the following rectangle, so that each side adds up to 18.

	8		
9	X	X	10
		7	

- (a) Express the state space of this problem as a set of constraints on the variables assigned to the different positions in the rectangle, where the following variable assignment is to be used:

x_1	8	x_2	x_3
9	X	X	10
x_4	x_5	7	x_6

SOLUTION:

The constraints in question may be sums of variables, inequalities and actual variable assignments. The Goal is an assignment of a number to each variable.

The set of equations is as follows:

From the rectangle:

(1) $x_1 + x_2 + x_3 = 10$

(2) $x_1 + x_4 = 9$

(3) $x_3 + x_6 = 8$

(4) $x_4 + x_5 + x_6 = 11$

From the problem statement:

(5) $x_i \neq x_j$ for $i \neq j$ where $i, j = 1, \dots, 6$

(6) $1 \leq x_1, \dots, x_6 \leq 6$

Derived:

(7) $3 \leq x_1, x_4 \leq 6$ from (2), (6) and problem statement

(8) $2 \leq x_3, x_6 \leq 6$ from (3), (6) and problem statement

- (b) Apply algorithm Backtrack (or Backtrack1) to find an assignment of numbers to variables which satisfies the problem statement. You can make only one variable assignment in each step (what will happen if you make more assignments?), but any variable assignment which follows directly from another variable assignment should be performed in the same step.

Number each state created. Indicate clearly which path is active at each point in time, and also indicate dead-end states and the number of the state returned to, if a dead-end state is encountered.

SOLUTION:

Backtrack is applied by performing a variable assignment, and for this assignment performing an additional assignment, etc, until either a consistent assignment for all variables is obtained or we have to backtrack and undo an assignment. Due to the nature of the problem, i.e., the existence of derived variables, we don't have to go down to

depth 6, but we can derive some dependent variables in each step. Note that additional relationships between variables may be derived, thus reducing the computations, e.g., $x_3, x_6 \neq 4$. However, facts such as this have not been taken advantage of, in order to demonstrate the backtracking process more clearly.

Assignment 1: $x_1 = 3 \Rightarrow x_4 = 6$ (from (2))

Recompute equations:

(1') $x_2 + x_3 = 7$

(4') $x_5 + x_6 = 5$

Assignment 1.1: $x_3 = 2 \Rightarrow x_6 = 6$ (from (3)) – DEADEND (from (5))

Assignment 1.2: $x_3 = 3$ – DEADEND (from (5))

Assignment 1.3: $x_3 = 4 \Rightarrow x_6 = 4$ (from (3)) – DEADEND (from (5))

Assignment 1.4: $x_3 = 5 \Rightarrow x_6 = 3$ (from (3)) – DEADEND (from (5))

Assignment 1.5: $x_3 = 6$ – DEADEND (from (5))

Backtrack – the first assignment has to be undone.

Assignment 2: $x_1 = 4 \Rightarrow x_4 = 5$ (from (2))

Recompute equations:

(1') $x_2 + x_3 = 6$

(4') $x_5 + x_6 = 6$

Assignment 2.1: $x_3 = 2 \Rightarrow x_6 = 6$ (from (3)) $\Rightarrow x_5 = 0$ (from (4')) – DEADEND (from (6))

Assignment 2.2: $x_3 = 3 \Rightarrow x_6 = 5$ (from (3)) – DEADEND (from (5))

Assignment 2.3: $x_3 = 4$ – DEADEND (from (5))

Assignment 2.4: $x_3 = 5$ – DEADEND (from (5))

Assignment 2.5: $x_3 = 6 \Rightarrow x_6 = 2$ (from (3)) $\Rightarrow x_5 = 4$ – DEADEND (from (5))

Backtrack – the second assignment has to be undone.

Assignment 3: $x_1 = 5 \Rightarrow x_4 = 4$ (from (2))

Recompute equations:

(1') $x_2 + x_3 = 5$

(4') $x_5 + x_6 = 7$

Assignment 3.1: $x_3 = 2 \Rightarrow x_6 = 6$ (from (3)) $\Rightarrow x_5 = 1$ (from (4')) $\Rightarrow x_2 = 3$ (from (1')) – SOLVED!!

(c) Draw the rectangle with the variable assignment you have found.

SOLUTION:

The rectangle with the final variable assignment is:

5	8	3	2
9	X	X	10
4	1	7	6

Exercise 3: Problem Formulation, Search

You have a 9×9 grid of squares, each of which can be coloured red or blue. The grid is initially coloured all blue, but you can change the colour of any square any number of times. Imagining that the grid is divided into nine 3×3 super-squares, you want each super-square to be all one colour but neighboring super-squares to be different colours.

- (a) Formulate this problem in the most straightforward way. Compute the size of the state space. If you were to use a search tree to find a solution, what would be the branching factor including and excluding revisits of ancestor states, and how many states would you generate in a tree of depth d ?

SOLUTION:

For each cell, there is a binary choice (R for red, B for blue). A state is a list of $9 \times 9 = 81$ cells (e.g., { B, B, R, B, ..., R }). In total there are 2^{81} distinct states.

- In every layer of the search tree, we can flip the colour of one of the 81 cells. Therefore, the branching factor is $b = 81$.
- After d layers of a search tree, we generate 81^d states, including revisits.
- If we exclude revisits of ancestor states, in the first layer we generate 81 states, in the second layer 81×80 states, and in layer d we generate $81 \times 80 \times \dots \times (81 - d + 1)$ states. So, the total number of states in a search tree of depth d is:

$$\sum_{i=1}^d \prod_{j=1}^i (81 - j + 1) = \sum_{i=1}^d \frac{81!}{(81 - i)!}$$

- (b) Given the goal, we need consider only colourings where each super-square is uniformly coloured. Reformulate the problem and compute the size of the state space. How many solutions does this problem have?

SOLUTION:

The formulation is the same as for (a), except now we have only 9 super-cells to colour. The state space comprises only 2^9 states.

This problem has two goal states, where a goal state is a checkerboard pattern of colours, starting either with red or with blue in the top-left corner, e.g.,

$$\begin{array}{ccc} \text{R} & \text{B} & \text{R} \\ \text{B} & \text{R} & \text{B} \\ \text{R} & \text{B} & \text{R} \end{array} \quad \text{or} \quad \begin{array}{ccc} \text{B} & \text{R} & \text{B} \\ \text{R} & \text{B} & \text{R} \\ \text{B} & \text{R} & \text{B} \end{array}$$

However, if you assume a non-zero path cost, only the second checkerboard is the solution, because it takes one fewer colouring step.

- (c) Part (b) has abstracted the original problem (a). Can you give a translation from the solution in (b) into a solution to problem (a)?

SOLUTION:

To transition from (b) to (a), we need to expand each cell in the state to the corresponding 9 cells. For example, if the solution to (b) lists cell { 1 }, then the solution to (a) lists { 1, 2, 3, 10, 11, 12, 19, 20, 21 }, and converts these cells to the same colour.

Exercise 4: Search Algorithms

- (a) Why can you stop BFS when you generate/find the goal, i.e., you don't need to wait for the goal to reach the top of the list in OPEN?

SOLUTION:

Because all the nodes generated later will have the same or greater cost than the cost of the nodes just generated.

- (b) How is DFS (or DLS) different from Backtrack?

SOLUTION:

Backtrack generates one child at a time, and deletes it when it backtracks. DFS generates all the children of a node, and deletes all of them when a solution has not been found down that path.

- (c) Which is more efficient if step costs are the same, BFS or UCS? Which is optimal if step costs are different?

SOLUTION:

When step costs are the same, BFS is more efficient, because it checks for the goal when a node is generated.

When step costs are different, UCS is optimal, while BFS does not guarantee optimality.

- (d) Modify the TreeSearch algorithm in Slide 31 of LN3, Part 1 to implement BFS.

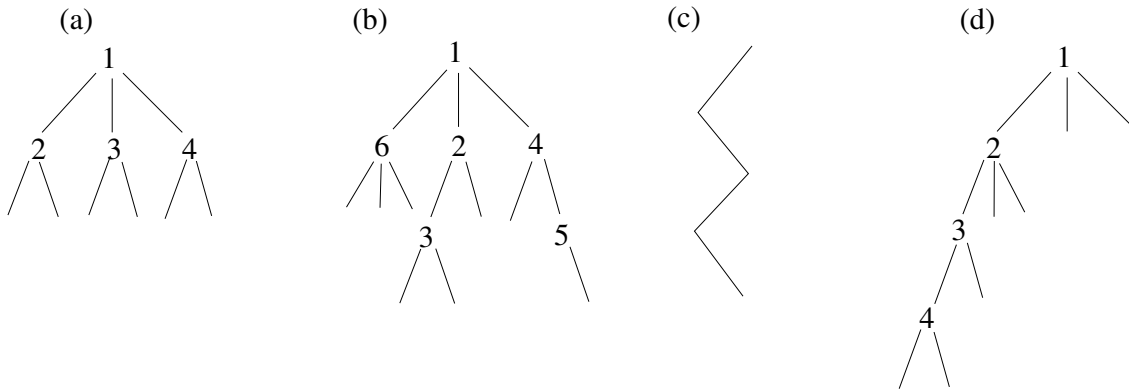
SOLUTION:

function TREESEARCH(*problem*) **returns** a solution or failure

- Initialize the *frontier* using the initial state of *problem*
- **Loop**
 1. **if** the *frontier* is $\emptyset$ **then** return failure
 2. **choose** a leaf node and remove it from the *frontier*
 3. **expand** the chosen node, **adding** the resultant nodes to the *frontier*
 4. **For each** resultant node
 - if** the node contains a goal state **then return** the corresponding solution

Exercise 5: Search Trees

Each of the following search structures has a distinctive shape and order of expansion which can be produced by a particular search procedure. For each tree write the name of the search procedure which can generate it — use the most specific option. The possible names are: backtrack, breadth first search, depth first search and tree search. Where applicable, the nodes are labeled with the order in which they are expanded.

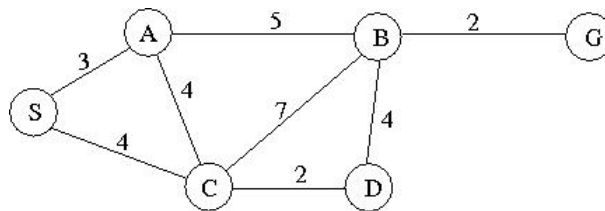


SOLUTION:

- (a) Breadth First Search, but it could also be generic tree search.
- (b) Tree search.
- (c) Backtrack, but it could also be Depth First Search (if each expansion has one option only).
- (d) Depth First Search, but it could also be generic tree search.

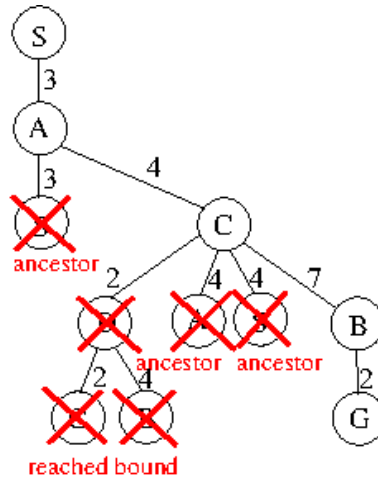
Exercise 6: Search Algorithms

The diagram below depicts the cost of travelling between cities.



- (a) Draw the search trajectory generated by the Backtrack procedure to reach the goal G starting from S , assuming that the BOUND for discontinuing the search is 4 steps. Use the heuristic *cheapest road* to order the rules. When backtracking, state clearly the reason for backtracking. What is the generated path? What is its cost?

SOLUTION:

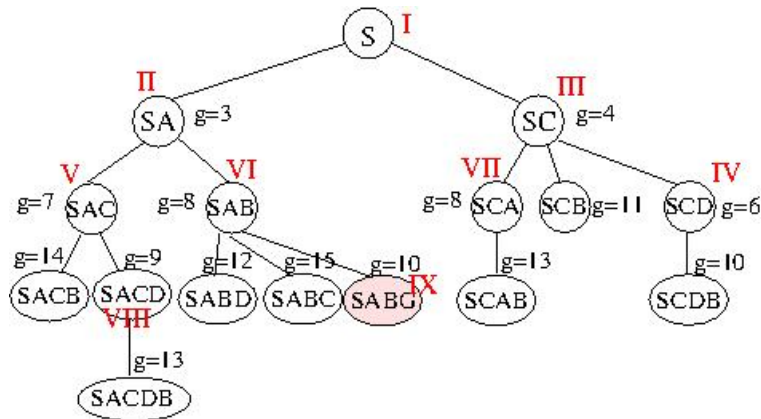


Total path length is $3+4+7+2=16$.

Path: S-A-C-B-G.

- (b) Draw the search *tree* generated by the Uniform-Cost Search (UCS) Tree search algorithm to reach the goal G starting from S . What is the generated path? What is its cost?

SOLUTION:



Iteration	Choose	Frontier
I	S	{S}
II	SA	{SA (3), SC (4)}
III	SC	{SC (4), SAC (7), SAB (8)}
IV	SCD	{SCD (6), SAC (7), SAB (8), SCA (8), SCB (11)}
V	SAC	{SAC (7), SAB (8), SCA (8), SCDB (10), SCB (11), SACB (14)}
VI	SAB	{SAB (8), SCA (8), SADC (9), SCDB (10), SCB (11), SABD (12), SACB (14), SABG (15)}
VII	SCA	{SACD (9), SABG (10), SCDB (10), SCB (11), SABD (12), SCAB (13), SACB (14), SABG (15)}
VIII	SACD	{SABG (10), SCDB (10), SCB (11), SABD (12), SACBD (13), SCAB (13), SACB (14), SABG (15)}
IX	SABG	{SCDB (10), SCB (11), SABD (12), SACBD (13), SCAB (13), SACB (14), SABG (15)}

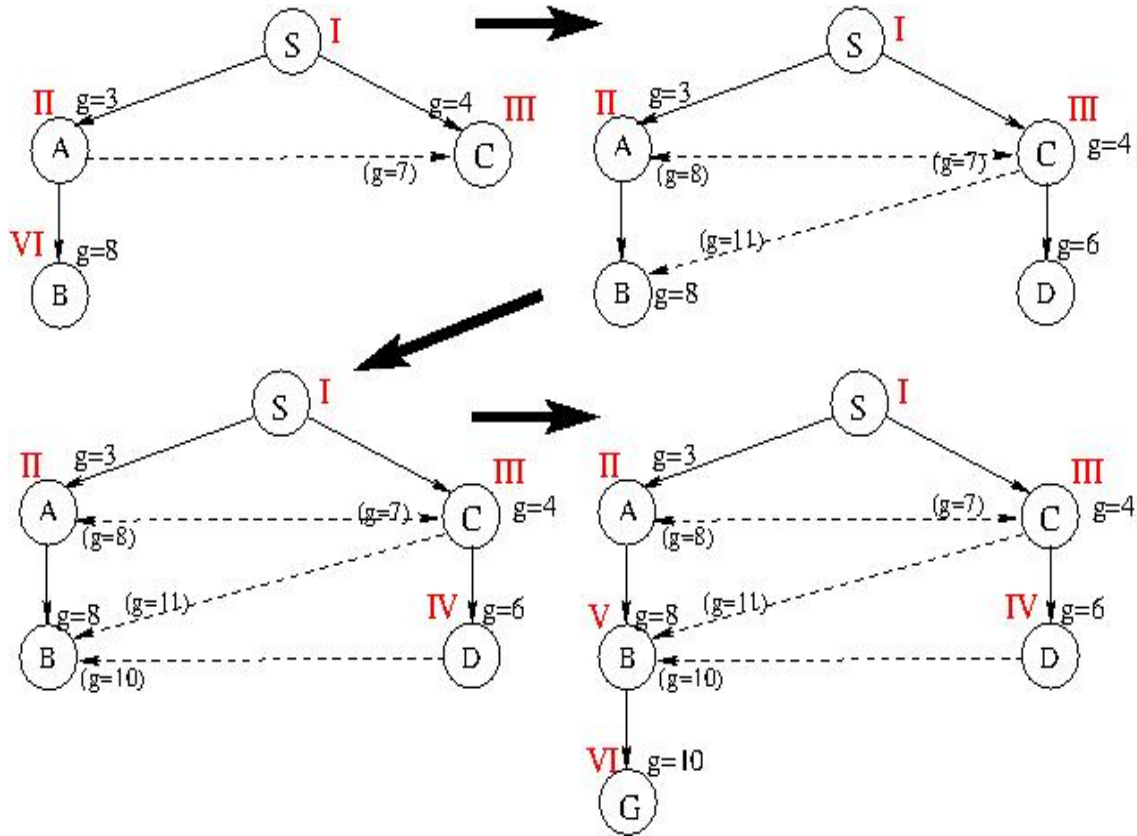
Total path length is 10.

Path: S-A-B-G.

IMPORTANT: Unlike BFS, we need to expand all the nodes whose cost is less than the cost of the goal node.

- (c) Draw the search *graph* generated by the UCS Graphsearch algorithm to reach the goal G starting from S . What is the generated path? What is its cost?

SOLUTION:



Iteration	Choose	Frontier	Explored
I	S	{S}	{S (0)}
II	A	{A (3), C (4)}	{S (0), A (3)}
III	C	{C (min{4,7}), B (8)}	{S (0), A (min{3,8}), C (min{4,7})}
IV	D	{D (6), B (min{8,11})}	{S (0), A (min{3,8}), C (min{4,7}), D (6)}
V	B	{B (min{8,11,10})}	{S (0), A (min{3,8}), C (min{4,7}), D (6), B (min{8,11,10})}
VI	G	{G (10)}	{S (0), A (min{3,8}), C (min{4,7}), D (6), B (min{8,11,10}), G (10)}